

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical
```

```
# Load dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data() # Load data
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ————— 3s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ————— 2s 0us/step
```

```
# Preprocessing
X_train = X_train.reshape(-1, 28, 28, 1) # Add channel dimension
X_test = X_test.reshape(-1, 28, 28, 1) # Add channel dimension

X_train = X_train / 255.0 # Normalize pixels
X_test = X_test / 255.0 # Normalize pixels

y_train = to_categorical(y_train, 10) # Convert labels to one-hot
y_test = to_categorical(y_test, 10) # Convert labels to one-hot
```

```
# Build CNN model
model = Sequential() # Initialize model

model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1))) # Conv layer
model.add(MaxPooling2D(2,2)) # Pooling layer

model.add(Conv2D(64, (3,3), activation='relu')) # Conv layer
model.add(MaxPooling2D(2,2)) # Pooling layer

model.add(Flatten()) # Convert to 1D

model.add(Dense(128, activation='relu')) # Fully connected layer
model.add(Dropout(0.5)) # Reduce overfitting

model.add(Dense(10, activation='softmax')) # Output layer
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# Compile model
model.compile(optimizer='adam', # Optimizer
              loss='categorical_crossentropy', # Loss function
              metrics=['accuracy']) # Evaluation metric
```

```
# Train model
history = model.fit(X_train, y_train, # Training data
                    epochs=5, # Number of iterations
                    batch_size=64, # Batch size
                    validation_split=0.1) # Validation data
```

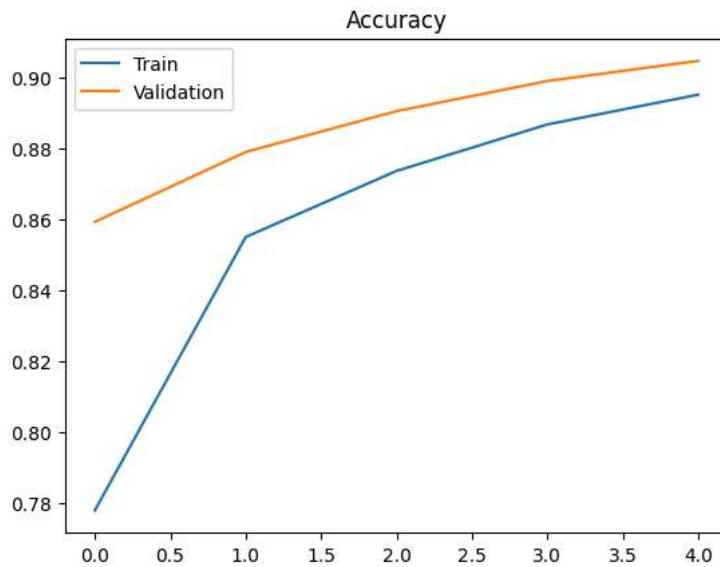
```
Epoch 1/5
844/844 ————— 54s 61ms/step - accuracy: 0.7781 - loss: 0.6161 - val_accuracy: 0.8593 - val_loss: 0.3888
Epoch 2/5
844/844 ————— 53s 63ms/step - accuracy: 0.8551 - loss: 0.4028 - val_accuracy: 0.8790 - val_loss: 0.3234
Epoch 3/5
844/844 ————— 50s 59ms/step - accuracy: 0.8737 - loss: 0.3495 - val_accuracy: 0.8905 - val_loss: 0.2998
```

```
Epoch 4/5
844/844 ————— 82s 59ms/step - accuracy: 0.8868 - loss: 0.3141 - val_accuracy: 0.8990 - val_loss: 0.2708
Epoch 5/5
844/844 ————— 48s 57ms/step - accuracy: 0.8951 - loss: 0.2881 - val_accuracy: 0.9047 - val_loss: 0.2537
```

```
# Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test) # Test performance
print("Test Accuracy:", test_acc) # Print accuracy
```

```
313/313 ————— 3s 10ms/step - accuracy: 0.8976 - loss: 0.2775
Test Accuracy: 0.897599995136261
```

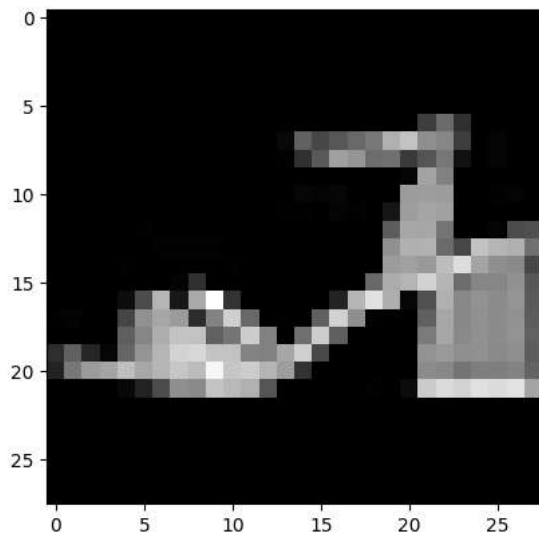
```
# Plot accuracy
plt.plot(history.history['accuracy']) # Train accuracy
plt.plot(history.history['val_accuracy']) # Validation accuracy
plt.legend(['Train', 'Validation']) # Legend
plt.title("Accuracy") # Title
plt.show() # Show plot
```



```
# Prediction
import random # Random selection

index = random.randint(0, len(X_test)-1) # Pick random image
sample = X_test[index] # Get image
```

```
plt.imshow(sample.reshape(28,28), cmap='gray') # Show image
plt.show() # Display image
```



```
prediction = model.predict(sample.reshape(1,28,28,1)) # Predict
print("Predicted:", np.argmax(prediction)) # Show result
```

1/1 — 0s 105ms/step
Predicted: 5